

Closing the Loop with Zephyr: Real-Time Flyback Control Using Zero-Latency Interrupts and CMSIS-DSP

S. Jaritz

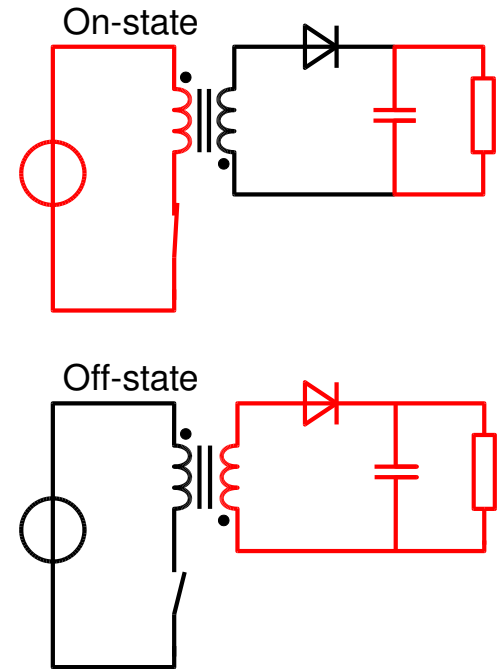
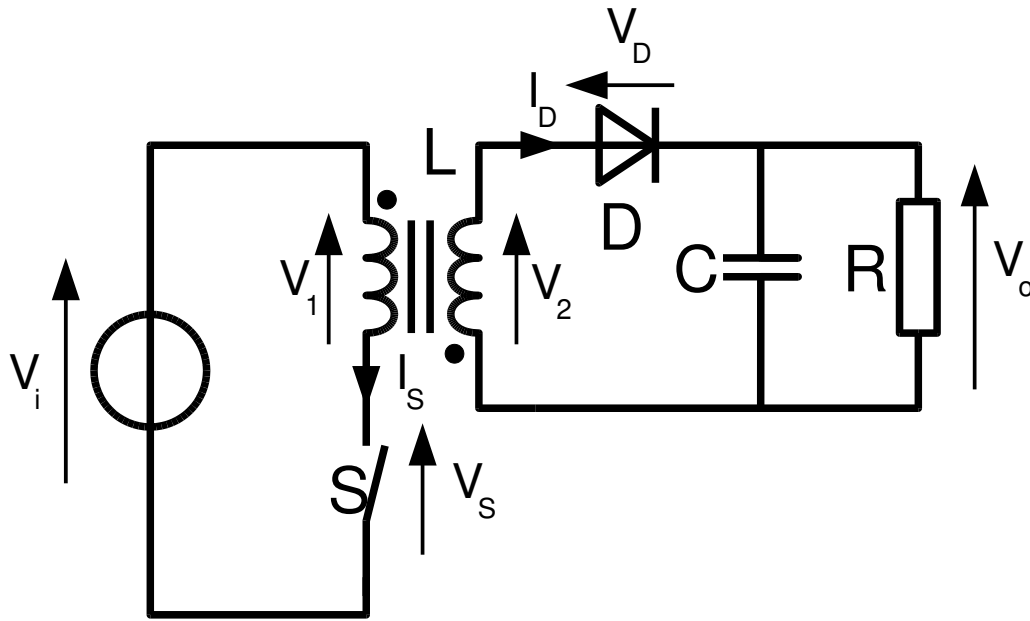
Zephyr Project Meetup: Vienna, Austria (17th June 2026)

The story

- **Problems with getting special IC in time of crisis**
- **Idea to use easy replaceable parts and off the shelf MCUs**
- **Applied to the real world problem of the PMU for embedded solutions**
- **Zephyr offers a great platform to connect and extend the PMU with new features and also offers the promise of less painful migration between MCUs**

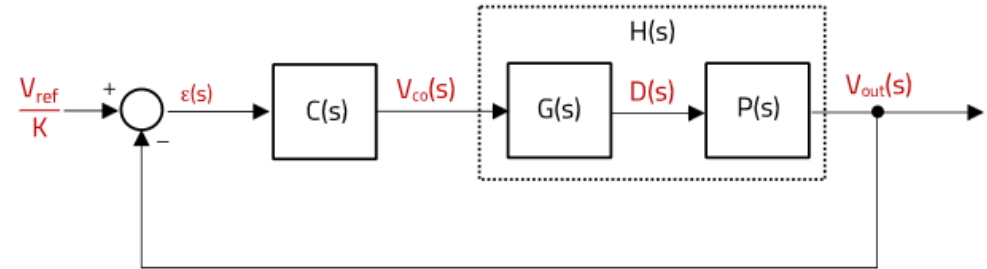
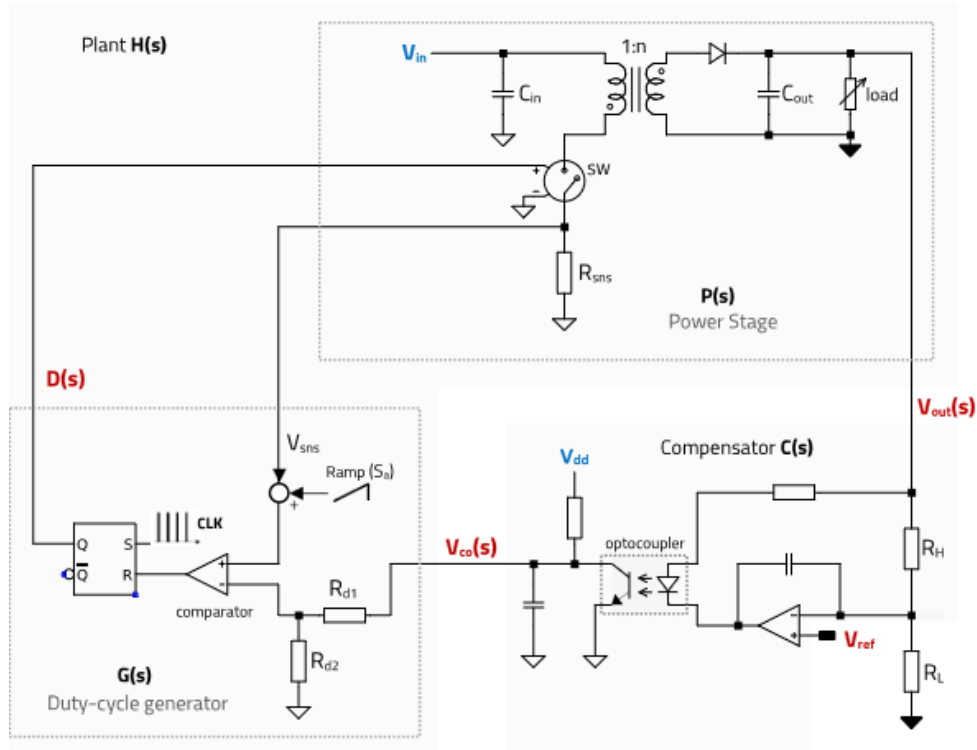
What is a flyback converter?

- AC/DC or DC/DC converter with galvanic isolation

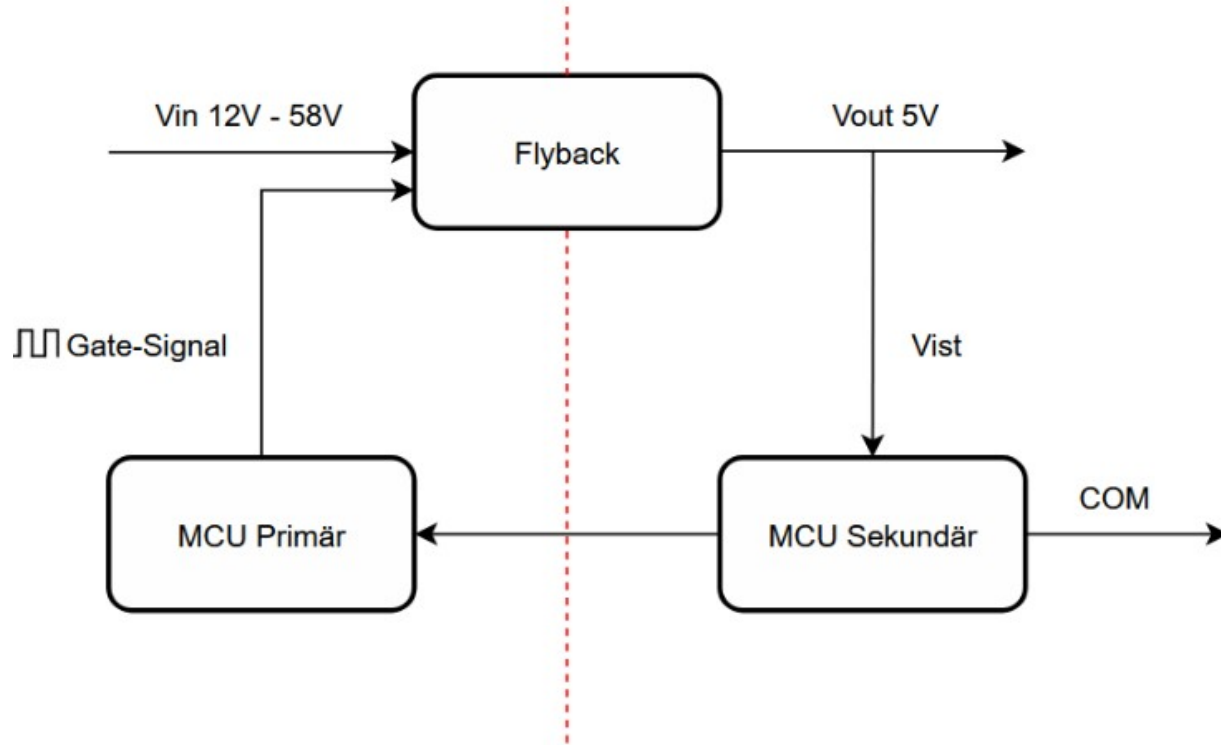


Picture source wikipedia

What is a real flyback converter?



Concept



Master Thesis: “Entwurf eines diskreten Wide-Input-Flyback-Konverters für langzeitstabile Industrieanwendungen” - Marco Stundner



Requirements for the firmware

- **Control the duty cycle by sampling the PWM feedback up to 300kHz**
- **Implement a PID controller**
- **Over/Under-current protection**
- **Basic signal filtering**

Our MCU

- **Pi 2350(with flash) – 1.1€**
 - 150MHz ARM Cortex-M33
 - PWM peripheral (generation and capture)
 - DMA, UART, I2C, ADC
- **$T_{\text{pwm}} = 1/3000000\text{s} = 3.33\mu\text{s}$**
- **$\text{Cycles} = 3.33\mu\text{s} * 150\text{MHz} = 500 \text{ cycles}$**

Some things we need to know

- **ISR based approach → PWM sampling, processing, PWM output → need highest prio**
- **ISR can be globally disabled (`irq_lock()` / `arch_irq_lock()`) → jitter**
- **Code that is run from flash goes through caching → cache optimisation**
- **Float32 and Q15 hardware support (with Q15 2-3x faster than float32)**

Design the signal path - ISR

- **CONFIG_ZERO_LATENCY_IRQS=y**
- **Low latency ISR with highest prio**
- **CONFIG_SRAM_VECTOR_TABLE=y**
- **Move ISR vector table to SRAM → avoid flash cache issue**

```
_ramfunc static void pwm_wrap_isr(const void *arg)
```

```
IRQ_DIRECT_CONNECT(PWM_IRQ_WRAP, 0, pwm_wrap_isr, 0);
```

```
irq_enable(PWM_IRQ_WRAP);
```

Design the processing 1/5

- **CONFIG_FPU=y**
- **CONFIG_CMSIS_DSP=y**
- **CONFIG_CMSIS_DSP_FILTERING=y**
- **CONFIG_CMSIS_DSP_CONTROLLER=y**
- **Q15 fixed point math**
- **PID controller and biquad based filter(IIR filter 2nd order)**

Design the processing 2/5

```
arm_pid_instance_q15 pid; // pid controller  
q15_t setpoint; // reference for the pid controller  
arm_biquad_casd_df1_inst_q15 lp; // low pass filter  
uint16_t y_prev; // y-1 sample
```

Design the processing 3/5

```
static inline q15_t filter(q15_t x)
{
    q15_t in = x;
    q15_t out;

    arm_biquad_cascade_df1_q15(
        &cntrl.lp,
        &in,
        &out,
        1
    );

    return out;
}
```

Design the processing 4/5

```
/* =====  
 * PWM DUTY UPDATE (10-90% clamp)  
 * ===== */  
  
static inline void update_pwm(uint32_t duty_ns)  
{  
    const uint32_t wrap = 1u << 16;  
  
    uint32_t level = (duty_ns * wrap) / PWM_PERIOD_NS;  
  
    /* enforce duty limits */  
    const uint32_t min_level = (uint32_t)(DUTY_MIN_FRAC * wrap);  
    const uint32_t max_level = (uint32_t)(DUTY_MAX_FRAC * wrap);  
  
    if (level < min_level) level = min_level;  
    if (level > max_level) level = max_level;  
  
    pwm_set_gpio_level(PWM_OUT_GPIO, level);  
}
```

Design the processing 5/5

```
__ramfunc static void pwm_wrap_isr(const void *arg)
{
    ARG_UNUSED(arg);
    q15_t y = read_feedback();
    q15_t y_delayed = (q15_t)(cntrl.y_prev << 3);
    cntrl.y_prev = y;
    q15_t y_f = filter(y_delayed);
    q15_t err = cntrl.setpoint - y_f;
    q15_t u = arm_pid_q15(&cntrl.pid, err);
    if (u > 25000) u = 25000;
    if (u < -25000) u = -25000;
    int32_t duty = ((int32_t)(u + 32768) * PWM_PERIOD_NS) >> 16;
    if (duty < 0) duty = 0;
    if (duty > (int32_t)PWM_PERIOD_NS) duty = PWM_PERIOD_NS;
    update_pwm((uint32_t)duty);
    pwm_clear_irq(cntrl.slice);
}
```

Result: Load and cycles

BEST CASE $\approx$ 128 cycles ($\sim 0.85 \mu\text{s}$ @ 150 MHz)

STD CASE $\approx$ 245 cycles ($\sim 1.63 \mu\text{s}$ @ 150 MHz)

WORST CASE $\approx$ 370 cycles ($\sim 2.47 \mu\text{s}$ @ 150 MHz)

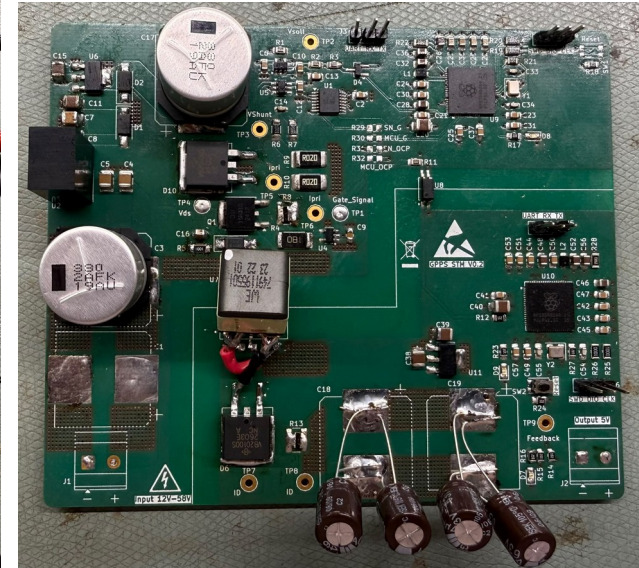
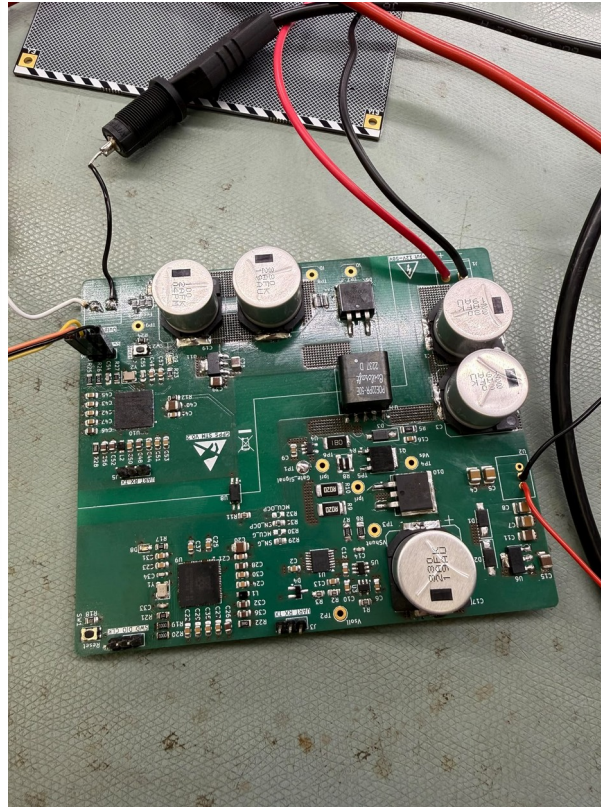
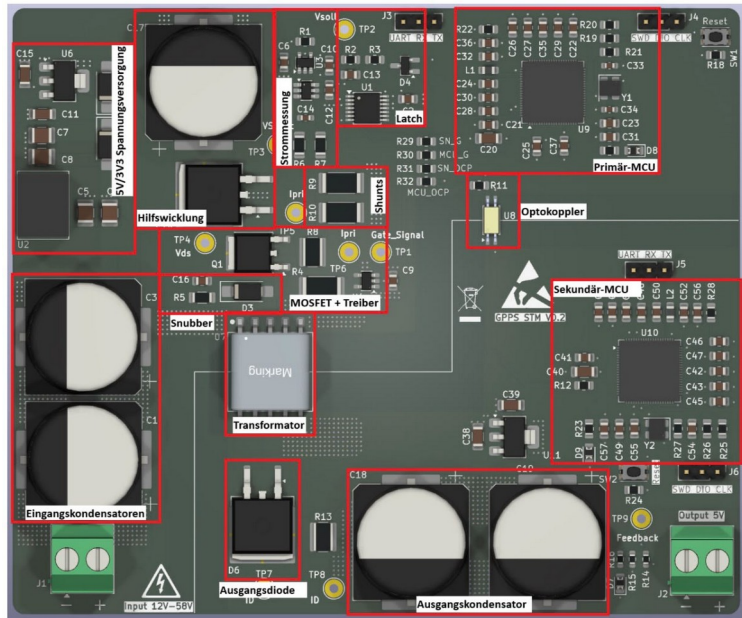
CPU LOAD @ 150 MHz:

BEST: $\sim 25\%$

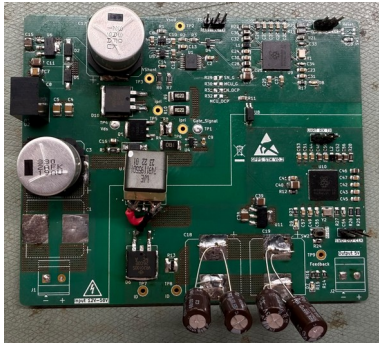
STD: $\sim 49\%$

WORST: $\sim 74\%$

PCB action



Magic Formula





Thanks - Let's talk!